

## 1. Problem Statement (From Your PDF)

👉 From page 1

### ● Problem in Simple Words

Wiki pages are interconnected (like links inside pages)

Example:

Page A → link → Page B → link → Page C

---

### 🚨 Issue

Normal RAG fails because:

#### ✗ 1. Context Gap

You retrieve Page A

But Page A has link to Page B

System does NOT read Page B

---

#### ✗ 2. Incomplete Answers

Important info is in linked pages

---

#### ✗ 3. No Relationship Understanding

Pages are connected

But RAG treats them as independent

---

### 🎯 Goal

Fetch context from linked (nested) pages

→ give complete answer

---

## 🧠 2. Three Approaches (From PDF)

---

### ● Approach 1: Metadata Enrichment

👉 From **page 3**

💡 **Idea**

Instead of full page → just add summary (metadata)

---

### **Example**

Page A has link to Page B

Instead of fetching B:

→ store summary of B

→ attach to A

---

### ✅ **Advantages**

✓ Fast

✓ Easy to implement

✓ Low cost

---

### ❌ **Disadvantages**

❌ Only summary (not full info)

❌ Shallow understanding

---

### ● **Approach 2: Graph RAG (IDEAL)**

👉 From **page 4**

💡 **Idea**

Build graph of pages

Use traversal to follow links

---

### **Example**

Page A → Page B → Page C

Query → start from A

→ go to B

→ go to C

---

### ✓ Advantages

- ✓ Deep understanding
- ✓ Multi-hop reasoning
- ✓ Best for complex queries

---

### ✗ Disadvantages

- ✗ Hard to build
- ✗ Needs graph infra
- ✗ More complexity

---

## ● Approach 3: Document Expansion

👉 From page 4–5

### 💡 Idea

At indexing time:

Merge linked pages into one document

---

### Example

Page A + Page B summary + Page C summary

→ store as one chunk

---

### ✓ Advantages

- ✓ Fast at query time
- ✓ No traversal needed

---

### ✗ Disadvantages

- ✗ Storage increases (2–5x)
  - ✗ Needs re-index if data changes
  - ✗ Chunk size issues
- 

### 3. Which Approach is BEST?

👉 From **page 6 (Recommended Path)**

🎯 **Answer:**

Graph RAG is BEST for final solution

---

#### **Why?**

Because:

- Handles links (relationships)
  - Supports multi-hop
  - Gives complete answers
- 

### **Recommended Strategy (VERY IMPORTANT)**

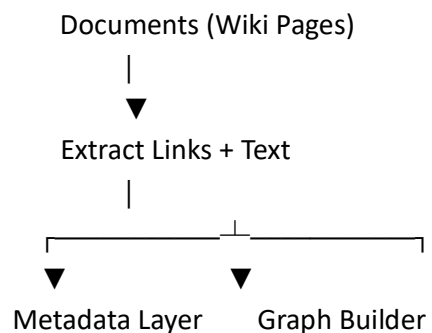
👉 From PDF:

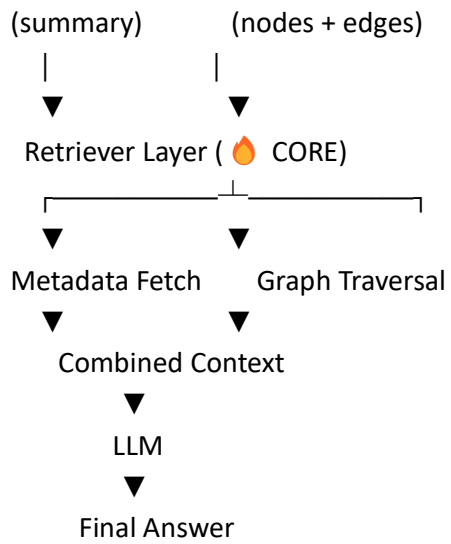
1. Start with Metadata (quick win)
  2. Move to Graph RAG (final solution)
  3. Use metadata as fallback
- 

### 4. BLUEPRINT (HOW TO BUILD)

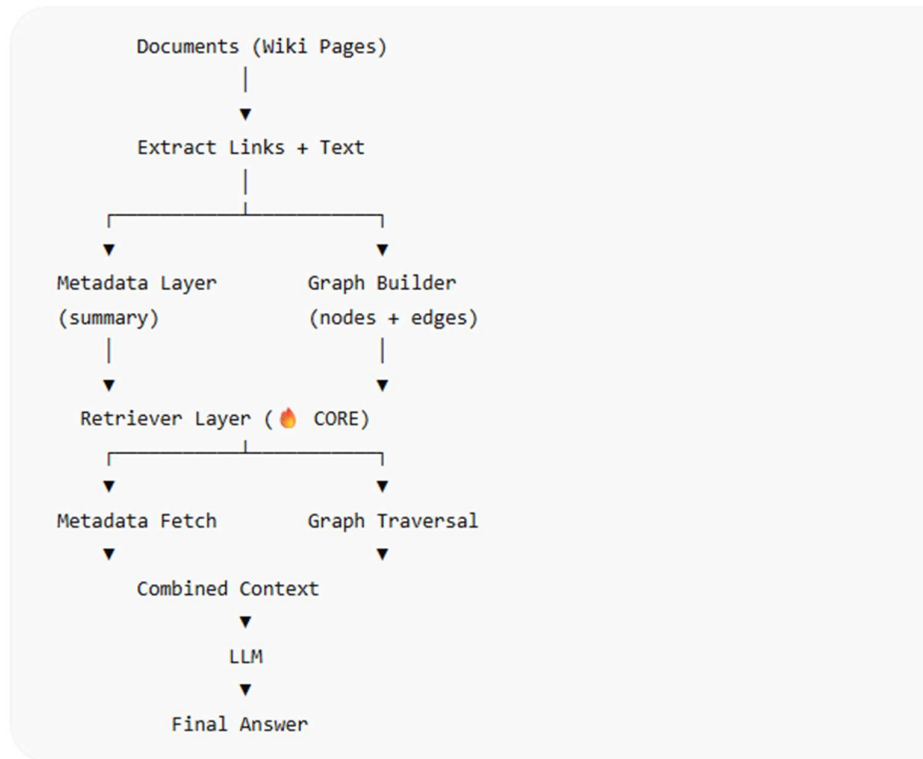
---

#### **FINAL ARCHITECTURE**





## 🔥 FINAL ARCHITECTURE



---

### 🧠 Retriever Logic

Query → find page → follow links → fetch related pages

---

## 5. SIMPLE CODE FLOW (Your Case)

---

### Step 1: Load Data

```
documents = load_wiki_pages()
```

---

### Step 2: Extract Links

```
links = extract_links(documents)
```

---

### Step 3: Build Graph

```
graph = build_graph(links)
```

---

### Step 4: Metadata Store

```
metadata = generate_summary(documents)
```

---

### Step 5: Retriever

```
def retrieve(query):  
    nodes = match_query(graph, query)  
    graph_context = traverse(graph, nodes)  
    meta_context = fetch_metadata(nodes)  
  
    return graph_context + meta_context
```

---

### Step 6: LLM

```
answer = llm(context, query)
```

---

## FINAL FLOW (VERY SIMPLE)

Query



Find page

↓

Follow links (Graph RAG 🔥 )

↓

Fetch metadata

↓

Combine context

↓

LLM

↓

Answer



## FINAL MEETING EXPLANATION

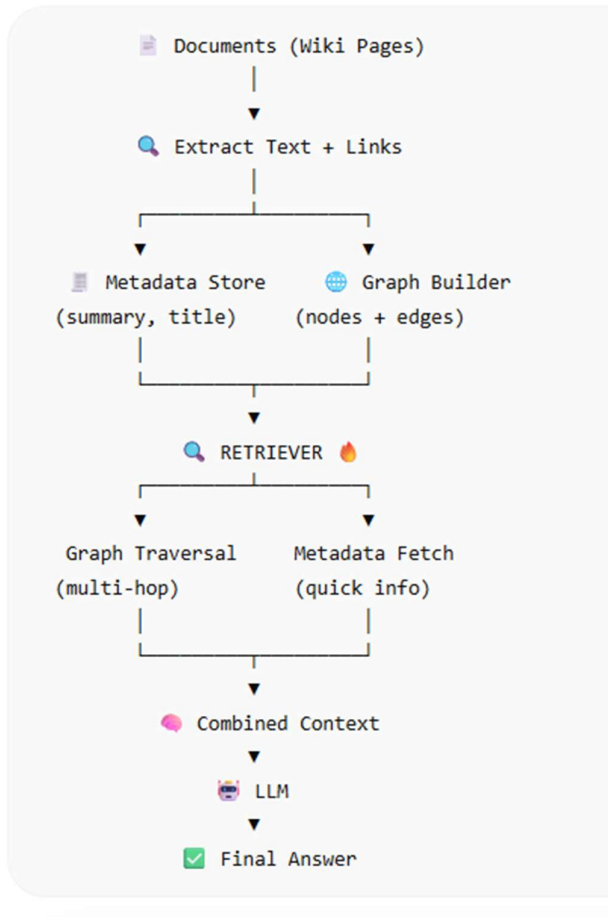
The problem is that wiki pages are interconnected, and simple RAG fails because it cannot follow links between documents. This leads to incomplete answers.

We evaluated three approaches: metadata enrichment, Graph RAG, and document expansion.

Metadata is fast but shallow, and document expansion increases storage. Graph RAG is the best approach because it captures relationships and supports multi-hop reasoning.

So, the recommended solution is to build a knowledge graph of documents and use graph traversal during retrieval, combined with metadata enrichment for efficiency. This ensures complete and accurate answers.

## SYSTEM FLOW



### ❓ Which Approach Should You Use?

👉 Based on your PDF:

Best REAL approach = Hybrid (Graph RAG + Metadata)

---

### 🎯 FINAL ANSWER

Use Graph RAG as MAIN

Use Metadata as SUPPORT

---

### 🧠 Why?

❌ Only Metadata

Fast but shallow ❌



## ❌ Only Graph

Powerful but heavy ❌

---

## ✅ Hybrid (BEST)

Graph → deep reasoning

Metadata → fast context

👉 Together:

Accurate + Fast + Complete ✅

---

## 🔥 FINAL FLOW (WHAT YOU SHOULD BUILD)

---

## 📊 SYSTEM FLOW

### tep-by-Step (HOW TO DO IT)

---

#### ● STEP 1: Data Ingestion

Load all wiki pages

```
documents = load_documents()
```

---

#### ● STEP 2: Extract Links

Find references between pages

Example:

Page A → Page B

```
links = extract_links(documents)
```

---

#### ● STEP 3: Build Graph (CORE)

Create connections between pages

```
graph.add_edge("Page A", "Page B")
```

---

#### ● STEP 4: Create Metadata

Store summary of each page

```
metadata["Page B"] = "short summary"
```

---

#### ● STEP 5: RETRIEVER (MOST IMPORTANT)

👉 This is your core logic

---

#### 🔥 What Retriever Does

1. Find matching page
  2. Traverse graph (multi-hop)
  3. Fetch linked pages
  4. Add metadata
  5. Combine context
- 

#### Example

Query:

"How combustion works?"

---

#### Graph Traversal:

Combustion → Fuel → Engine

---

#### Metadata:

Fuel → "Fuel is used for energy"

---

#### Final Context:

Combustion → Fuel → Engine  
+ metadata summaries

---

## STEP 6: LLM

```
answer = llm(context, query)
```

---

## SIMPLE RETRIEVER CODE (YOUR CASE)

```
def hybrid_retriever(graph, metadata, query):

    nodes = []

    # Step 1: match nodes
    for node in graph.nodes():
        if node.lower() in query.lower():
            nodes.append(node)

    context = []

    # Step 2: graph traversal
    for node in nodes:
        neighbors = nx.single_source_shortest_path_length(graph, node, cutoff=2)

        for n in neighbors:
            for nb in graph.neighbors(n):
                context.append(f"{n} → {nb}")

    # Step 3: metadata
    for node in nodes:
        if node in metadata:
            context.append(metadata[node])

    return "\n".join(context)
```

---

## WHY THIS APPROACH IS BEST

---

### Handles Problem from PDF

From your PDF:

Problem: nested links ❌

---

### ✅ Your Solution

Graph → follows links

Metadata → adds quick context

---

### ✔ Solves All Issues

| Problem | Solution |
|---------|----------|
|---------|----------|

|             |                 |
|-------------|-----------------|
| Context gap | Graph traversal |
|-------------|-----------------|

|              |           |
|--------------|-----------|
| Missing info | Multi-hop |
|--------------|-----------|

|               |       |
|---------------|-------|
| Relationships | Graph |
|---------------|-------|

|       |          |
|-------|----------|
| Speed | Metadata |
|-------|----------|

---

### 🎯 FINAL ONE-LINE (MEETING)

We use a hybrid Graph RAG approach where we build a knowledge graph to traverse linked documents and combine it with metadata enrichment to efficiently retrieve complete and connected context for accurate answer generation.

---

### 🚀 What You Should Say Confidently

Graph RAG is the main approach because it solves the link problem using multi-hop traversal, and metadata is used as a lightweight layer to improve retrieval speed and context completeness.

---

### 🔥 YOU ARE NOW DOING

Production-level RAG design ✅

## SIMPLE SUMMARY — HOW TO DO IT

---

### Goal

Documents are connected (linked pages)  
We need to follow links and give full answers

---

### FINAL APPROACH (VERY SIMPLE)

Use:

-  Graph RAG (main)
  -  Metadata (support)
- 

### HOW TO DO (STEP-BY-STEP)

---

#### Step 1: Read Documents

Load all wiki pages

---

#### Step 2: Find Links

Page A → Page B

Page B → Page C

-  Extract these connections
- 

#### Step 3: Build Graph

Make graph:

A → B → C

-  This helps you follow links
- 

#### Step 4: Create Metadata

For each page:  
store summary

Example:

Fuel page → "Fuel is used for combustion"

---

### **Step 5: Retriever (MOST IMPORTANT)**

When user asks:

"How combustion works?"

 Do this:

1. Find node → Combustion
  2. Follow links → Fuel → Engine
  3. Get metadata
  4. Combine everything
- 

### **Step 6: Send to LLM**

Give combined context → LLM → Answer

---

### **FINAL FLOW (VERY EASY)**

Query

↓

Find page

↓

Follow links (Graph )

↓

Get summaries (Metadata)

↓

Combine

↓

LLM

↓

Answer

---

### **WHY THIS WORKS**

Graph → follows links (deep understanding)

Metadata → gives quick info (fast)

👉 Together:

Complete + Fast + Accurate ✅

---

### 🚀 ONE-LINE (MEETING)

We build a graph of documents to follow links using multi-hop traversal and combine it with metadata summaries to retrieve complete context before passing it to the LLM.

---

### 🔥 SUPER SIMPLE ANALOGY

Like Wikipedia:

You read one page

→ click links

→ read related pages

→ understand full topic

👉 That's exactly what your system does 😊

---

### ✅ DONE

Now you can explain:

✓ Problem

✓ Solution